

19 Testing

Die erstellten Funktionen werden mit Hilfe des Programmes «Postman» getestet. Postman eignet sich perfekt um API Schnittstellen zu testen. Da die User Übergaben einfach simuliert und geändert werden können, ist dies die einfachste Variante die verschiedenen Situationen auf deren Verhaltensweise zu testen.

Für die folgenden Tests wird die Version 7.20.1 von Postman verwendet.

Damit die einzelnen Files untereinander korrekt referenziert sind, und somit miteinander kommunizieren können, wird die Ordnerstruktur welche auf Abbildung 41 zu sehen ist, verlangt.

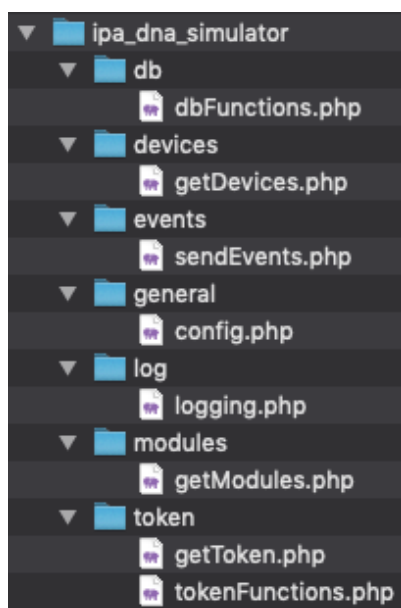


Abbildung 41: Ordnerstruktur programmierter Scripts

Alle Files sind im Anhang hinterlegt. Somit können alle Tests, wenn gewünscht nachgetestet werden.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **86** von **129**

19.1 Auflistung Testfälle

In der Tabelle 21 sind alle Testfälle aufgeführt, welche die Funktionalität der erstellten Simulation sicherstellt.

Test ID	Aktivität
T-01	Token generieren mit korrekter User Übergabe
T-02	Token generieren mit inkorrekt oder fehlender User Übergabe
T-03	Device Abfrage mit korrektem Token
T-04	Device Abfrage mit falschem Token Format
T-05	Device Abfrage mit abgelaufenem Token
T-06	Device Abfrage mit korrekten Parametern
T-07	Device Abfrage mit inkorrekten Parametern
T-08	Modul Abfrage ohne Angabe der deviceId
T-09	Modul Abfrage mit einer falschen deviceId
T-10	Modul Abfrage ohne Parameter (ausser deviceId)
T-11	Modul Abfrage mit korrekten Parametern
T-12	Modul Abfrage mit inkorrekten Parametern (Parameter widersprechen sich)
T-13	Events können gesendet werden (ohne Parameter)
T-14	Events können gesendet werden (mit Parameter)

Tabelle 21: Auflistung Testfälle

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **87** von **129**

19.2 Ausführung / Durchführung Testfälle

Für jeden aufgeführten Testfall werden der Ablauf und die Voraussetzungen des Tests festgehalten. Da jede Abfrage zum Schluss geloggt wird, wird dies einfachheitshalber nicht jedes Mal aufgeschrieben.

Auf der Abbildung 42 und der Abbildung 43 ist kurz erklärt, welche Anpassungen für das Nachtesten der Funktionen, erforderlich sein können.

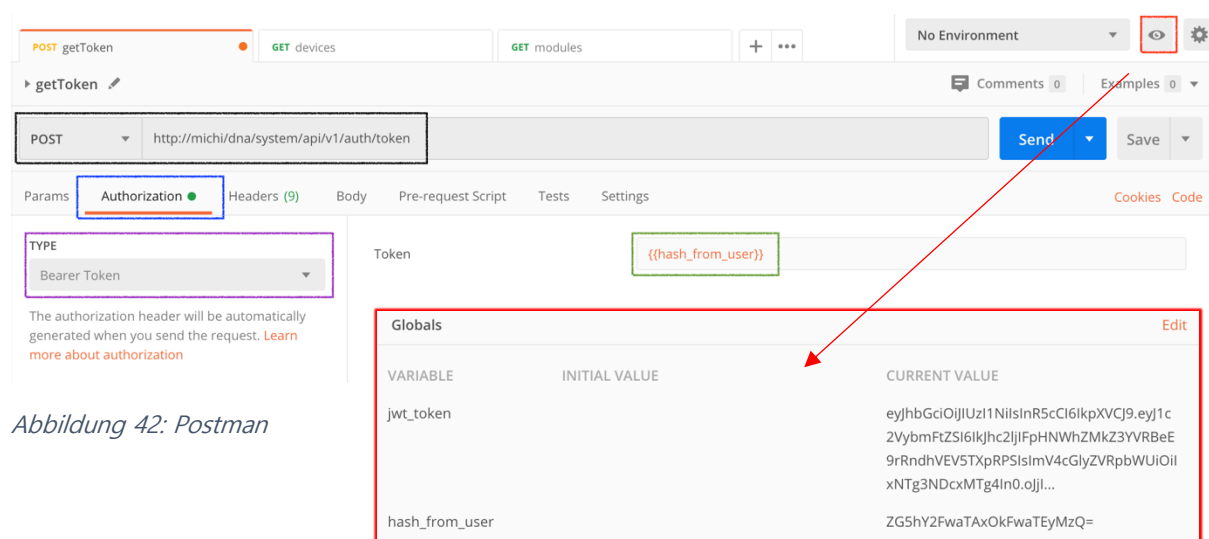


Abbildung 42: Postman

Abbildung 43: Postman Globals

- Über den blau umrahmten Reiter kann die Autorisierung eingestellt werden.
- Im violett umrahmten Dropdown Menü ist der Typ «Bearer Token» auszuwählen.
- Globale Variablen können im Postman über das rot markierte «Augensymbol» hinterlegt werden.
- Der Name der Variable wird im grün umrahmten Feld mit zwei geschweiften Klammern eingetragen.
- Die URL und die Methode der Abfrage wird im schwarz umrahmten Feld eingetragen

Die 3 URL der API Schnittstellen, welche für die Ausführung der Tests notwendig sind, sind auf Seite 64 abgebildet.

T-01	Token generieren mit korrekter User Übergabe
Testszenario	Ein Token wird generiert
Testvoraussetzung	Die User Angaben werden vollständig und im korrekten Format übergeben.
Testschritte	- Die Userangaben mithilfe eines Base64 Kodierer online encoden. (https://www.base64encode.org/) - Format: (username:password) - Enkodierter Hash im Postman als Globale Variable namens «hash_from_user» eintragen. - Der Typ der Autorisation «Bearer Token» setzen. - Abfrage senden
Erwartete Resultate	Ein JWT wird generiert und in der Datenbank eingetragen. Bei jedem weiteren Aufruf der API wird ein neuer Token generiert und in der Datenbank aktualisiert. Auch die Ablaufzeit des Tokens wird jedes Mal aktualisiert.
Output <pre> 1. { 2. "Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Ikhjc2ljIFpHNWZhZMkZ3YVRBeE9rRndhVEV5TXpRPStImV4cGlyZVRpbWUiOiIxNTg3MzY5MzczIn0.GNk17Eley7X2x1707Q_EHn3IbdiFB1xjotnvWr4pb0w" 3. }</pre>	
Teststatus	ERFOLGREICH

Tabelle 22: Ausführung T-01

T-02	Token generieren mit inkorrekt oder fehlender User Übergabe
TestszENARIO	Die Token Abfrage wird ohne oder mit falschen User Angaben aufgerufen.
Testvoraussetzung	Die User Übergabe ist falsch oder fehlt komplett.
Testschritte	1. User Angaben aus den Globalen Variablen entfernen oder verfälschen 2. Abfrage senden
Erwartete Resultate	Es wird kein Token generiert.
Output <pre> 1. { 2. "error": "Authentication has failed. Please provide valid credentials." 3. }</pre>	
Teststatus	ERFOLGREICH

Tabelle 23: Ausführung T-02

T-03	Device Abfrage mit korrektem Token
TestszENARIO	Die API für das Device Inventar wird mit korrektem, gültigem Token aufgerufen. Parameter werden keine verwendet.
Testvoraussetzung	Der Token wurde vor weniger als einer Stunde generiert. Der Token wurde aus gültigen User Angaben generiert.
Testschritte	- Der generierte Token wird als Globale Variable namens «jwt_Token» im Postman erfasst. (Dies simuliert die Übergabe des Tokens durch den User) - Der Typ der Autorisation «Bearer Token» setzen. Abfrage senden
Erwartete Resultate	Eine Liste aller Geräte wird im korrekten Format ausgegeben.
Output - Es wird die Liste aller Devices ausgegeben. Da diese deutlich zu lang ist um hier einzufügen, wird dies unterlassen. -	
Teststatus	ERFOLGREICH

Tabelle 24: Ausführung T-03

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **90** von **129**

T-04	Device Abfrage mit falschem Token
Testszenario	Die API für das Device Inventar wird mit falschem, ungültigem Token aufgerufen. Parameter werden keine verwendet.
Testvoraussetzung	Der Token ist falsch und erfüllt die Token Autorisierung nicht.
Testschritte	- Die Globale Variable des Tokens im Postman wird mit beliebigen Werten verändert.
Erwartete Resultate	Es werden keine Geräte ausgegeben. Es erscheint eine Fehlermeldung, die besagt, dass ein Fehler bei der Autorisierung aufgetreten ist.
Output 401 - The client's authentication credentials included with the request are missing or in valid.	
Teststatus	ERFOLGREICH

Tabelle 25: Ausführung T-04

T-05	Device Abfrage mit abgelaufenem Token
Testszenario	Die API für das Device Inventar wird mit abgelaufenem, ungültigem Token aufgerufen. Parameter werden keine verwendet.
Testvoraussetzung	Der Token ist formal in korrektem Zustand. Der Token ist allerdings nicht mehr gültig, da er seine Lebenszeit überschritten hat.
Testschritte	1. Damit man keine Stunde warten muss, um diesen Test durchführen zu können, setzt man im Konfigurationsfile den Wert der Token Lebensdauer auf 20s. 2. Die API der Token Generation wird ausgeführt. 3. Der generierte Token wird in die Globale Variable im Postman geschrieben. 4. Wenn mindestens 20s vergangen sind, seit dem der Token generiert wurde, wird die Device Abfrage gesendet.
Erwartete Resultate	Es werden keine Geräte ausgegeben. Es erscheint eine Fehlermeldung, die besagt, dass ein Fehler bei der Autorisierung aufgetreten ist.
Output {"message": "Unauthorized"}	
Teststatus	ERFOLGREICH

Tabelle 26: Ausführung T-05

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **91** von **129**

T-06	Device Abfrage mit korrekten Parametern
Testszenario	Die API für das Device Inventar wird mit korrektem, gültigem Token aufgerufen. Als Parameter können verschiedene Kombinationen mitgegeben werden.
Testvoraussetzung	Wenn die id als Parameter mitgegeben wurde, ist dies der bestimmende Parameter. Die Weiteren werden also nicht beachtet. Wenn die id nicht angegeben wurde, müssen die weiteren Parameter übereinstimmen, um ein Output zu erzeugen.
Testschritte	1. Id als Parameter mitgeben. Dazu folgender String an die URL hängen: «?id=075e39d1-976a-49e8-ab41-3ee908d504ef» 2. Id und passender Hostname als Parameter mitgeben. Dazu folgender String an die URL hängen: «?id=075e39d1-976a-49e8-ab41-3ee908d504ef&hostname=Edge_3.sg.labnet.local» 3. Id und Hostname passen nicht überein. Dazu folgender String an die URL hängen: «?id=075e39d1-976a-49e8-ab41-3ee908d504ef&hostname=falscher_hostname»
Erwartete Resultate	Fall 1: Es wird nur das Gerät mit der gewünschten Id im korrekten Format ausgegeben. Fall 2: Es wird nur das Gerät mit der gewünschten Id und dem passenden Hostname im korrekten Format ausgegeben. Fall 3: Es wird nur das Gerät mit der gewünschten Id im korrekten Format ausgegeben.
Output Fall 1: <pre>{ "response": [{ "id": "075e39d1-976a-49e8-ab41-3ee908d504ef", "hostname": "Edge_3.sg.labnet.local", "managementIpAddress": "192.168.8.98", "macAddress": "70:1f:53:01:25:80", "locationName": "None", "serialNumber": "FCW2135L0MU", "location": "None", "collectionStatus": "Managed", "errorCode": "None", "softwareVersion": "16.9.3s", "softwareType": "IOS-XE", "role": "ACCESS", "reachabilityStatus": "Reachable", "associatedWlcIp": null, "type": null, "series": null, "platformId": null, "family": null, "description": null, "apManagerInterfaceIp": "10.44.1.213", "bootDateTime": null, "collectionInterval": "NA", "errorDescription": "are occurring with this device. Either the SNMP credentials are not correctly provided to controller or the device is responding slow and SNMP timeout is low. If it is a timeout issue, controller will attempt to progressively adjust the timeout in subsequent collection cycles to get device to managed state. User can also run discovery again only for this device using the discovery feature after adjusting the timeout and SNMP credentials as required. Or user can update the timeout and SNMP credentials as required using update credentials", "instanceTenantId": "5b9f5b41056a300088e7bdde", "instanceUuid": "075e39d1-976a-49e8-ab41-3ee908d504ef", "interfaceCount": "0", "inventoryStatusDetail": "NA", "lastUpdateTime": "1583942792872", "lastUpdated": "2020-03-" }] }</pre>	

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **92** von **129**

```
11 16:06:32", "lineCardCount": "0", "lineCardId": null, "memorySize": "NA", "reachabilityFailureReason": "NA", "roleSource": "AUTO", "snmpContact": "null", "snmpLocation": "default location", "tagCount": "0", "tunnelUdpPort": "16666", "upTime": "11days 19:28:49.850", "waasDeviceMode": "null"}, {"version": "1.0"}]
```

Fall 2:

- Wie bei Fall 1

Fall 3:

- Wie bei Fall 1

Teststatus

ERFOLGREICH

Tabelle 27: Ausführung T-06

T-07	Device Abfrage mit inkorrekten Parametern
TestszENARIO	Die API für das Device Inventar wird mit korrektem, gültigem Token aufgerufen. Allerdings wird ein unbekannter, inkorrekt Parameter eingegeben.
Testvoraussetzung	Der Token ist korrekt und gültig.
Testschritte	1. Unbekannter Hostname als Parameter mitgeben: URL Zusatz = «?hostname=nichtexistent»
Erwartete Resultate	Es wird kein Gerät ausgegeben. Die Abfrage ist erfolgreich, liefert allerdings keinen Output.
Output	
{"response": [], "version": "1.0"}	
Teststatus	ERFOLGREICH

Tabelle 28: Ausführung T-07

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **93** von **129**

T-08	Modul Abfrage ohne Angabe der deviceId
TestszENARIO	Die Modul Abfrage wird ohne Angabe der deviceId aufgerufen.
Testvoraussetzung	Der Token ist korrekt und gültig.
Testschritte	1. URL für die Modul Abfrage mit korrektem Token eingeben. 2. Parameter «deviceId» fehlt komplett
Erwartete Resultate	Da ein Modul nur mit der deviceId aufgerufen werden kann, da das Modul nur so einem Device zugeordnet werden kann, ist dieser Parameter erforderlich. Wird dieser nicht mitgegeben, wird kein Modul Output generiert.
Output <pre>{ "response": { "errorCode": "Bad request", "message": "Invalid input request", "detail": "Required String parameter \"deviceId\" is not present" }, "version": "1.0" }</pre>	
Teststatus	ERFOLGREICH

Tabelle 29: Ausführung T-08

T-09	Modul Abfrage mit einer falschen deviceId
TestszENARIO	Die Modulabfrage wird mit einer falschen deviceId gesendet. Dabei kann kein Modul identifiziert werden.
Testvoraussetzung	Der Token ist korrekt und gültig. Der Parameter deviceId wurde mitgegeben.
Testschritte	1. URL für die Modul Abfrage mit korrektem Token eingeben. 2. Als Parameter für die «deviceId» wird ein falscher Wert eingegeben. URL Zusatz = «?deviceId=12345nichtexistent12345»
Erwartete Resultate	Kein Modul Output. Fehlermeldung mit Hinweis zur eingegebenen Id wird ausgegeben.
Output <pre>204 - The request was successful, however no content was returned</pre>	
Teststatus	FEHLGESCHLAGEN Das Output Format ist nicht korrekt. Zudem muss ein Bezug zur eingegebenen Id ersichtlich sein.

Tabelle 30: Ausführung T-09

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **94** von **129**

T-10	Modul Abfrage ohne Parameter (ausser deviceId)
Testszenario	Die Modul Abfrage wird richtig abgefragt.
Testvoraussetzung	Der Token ist korrekt und gültig. Der Parameter deviceId wurde mitgegeben und ist gültig.
Testschritte	1. Ausführen einer korrekten Abfrage. URL Zusatz = «?deviceId=075e39d1-976a-49e8-ab41-3ee908d504ef»
Erwartete Resultate	Alle Module, die der eingegebenen deviceId zugeordnet sind, werden im korrekten Format angezeigt.
Output <pre>{ "response": [{ "id": "0cbfc068-07ed-44d7-89d5-7365f0b25188", "isReportingAlarmsAllowed": "1", "serialNumber": "114-7745-33", "manufacturer": "1237", "typeId": "1", "name": "Switch 1 FRU Uplink Module 1", "description": "8x10G Uplink Module", "vendorEquipmentType": "cevModuleC9300NM8X10G", "partNumber": "C9300-NM-8X", "isFieldReplaceable": "1" }, { "id": "1234", "isReportingAlarmsAllowed": "1", "serialNumber": "12345", "manufacturer": "cisco", "typeId": "1", "name": "Switch 1 FRU Uplink Module 1", "description": "8x10G Uplink Module", "vendorEquipmentType": "cevModuleC9300NM8X10G", "partNumber": "C9300-NM-8X", "isFieldReplaceable": "1" }, { "id": "123456", "isReportingAlarmsAllowed": "1", "serialNumber": "123456789", "manufacturer": "cisco2", "typeId": "2", "name": "Switch 1 FRU Uplink Module 1", "description": "4x 1G/10G network module", "vendorEquipmentType": "cevModuleC9200NM4X10G", "partNumber": "C9200-NM-4X", "isFieldReplaceable": "1" }, { "id": "2b170138-d0e6-42e5-8b51-7cb7c15c0965", "isReportingAlarmsAllowed": "1", "serialNumber": "1234", "manufacturer": "1234", "typeId": "1", "name": "Switch 1 FRU Uplink Module 1", "description": "8x10G Uplink Module", "vendorEquipmentType": "cevModuleC9300NM8X10G", "partNumber": "C9300-NM-8X", "isFieldReplaceable": "1" }], "version": "1.0" }</pre>	
Teststatus	ERFOLGREICH

Tabelle 31: Ausführung T-10

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **95** von **129**

T-11	Modul Abfrage mit korrekten Parametern
Testszenario	Die Modulabfrage wird mit richtigen Parametern ausgeführt.
Testvoraussetzung	Der Token ist korrekt und gültig. Der Parameter deviceId wurde mitgegeben und ist gültig.
Testschritte	1. Ausführen einer korrekten Abfrage mit Limit. URL Zusatz = «?deviceId=075e39d1-976a-49e8-ab41-3ee908d504ef&limit=2» 2. Ausführen einer korrekten Abfrage mit Limit und Offset. URL Zusatz = «?deviceId=075e39d1-976a-49e8-ab41-3ee908d504ef&limit=2&offset=1» 3. Ausführen einer korrekten Abfrage gefiltert nach der «partNumber» URL Zusatz = «?deviceId=075e39d1-976a-49e8-ab41-3ee908d504ef&partNumberList=C9300-NM-8X»
Erwartete Resultate	Fall 1: Es werden nur 2 Module angezeigt. Fall 2: Es werden nur 2 Module ausgegeben. Allerdings mit Offset 1. (Also Modul 2 und 3, das gefunden wird.) Fall 3: Es werden nur die Module der entsprechenden Id und der passenden partNumber angezeigt.

Output

Fall 1:

```
{
  "response": [
    {
      "id": "0cbfc068-07ed-44d7-89d5-7365f0b25188",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "114-7745-33",
      "manufacturer": "1237",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    },
    {
      "id": "1234",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "12345",
      "manufacturer": "cisco",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    }
  ],
  "version": "1.0"
}
```

Fall 2:

```
{
  "response": [
    {
      "id": "1234",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "12345",
      "manufacturer": "cisco",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    },
    {
      "id": "123456",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "123456789",
      "manufacturer": "cisco2",
      "typeId": "2",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "4x 1G/10G network module",
      "vendorEquipmentType": "cevModuleC9200NM4X10G",
      "partNumber": "C9200-NM-4X",
      "isFieldReplaceable": "1"
    }
  ],
  "version": "1.0"
}
```

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: 96 von 129

Fall 3:

```
{
  "response": [
    {
      "id": "0cbfc068-07ed-44d7-89d5-7365f0b25188",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "114-7745-33",
      "manufacturer": "1237",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    },
    {
      "id": "1234",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "12345",
      "manufacturer": "cisco",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    },
    {
      "id": "2b170138-d0e6-42e5-8b51-7cb7c15c0965",
      "isReportingAlarmsAllowed": "1",
      "serialNumber": "1234",
      "manufacturer": "1234",
      "typeId": "1",
      "name": "Switch 1 FRU Uplink Module 1",
      "description": "8x10G Uplink Module",
      "vendorEquipmentType": "cevModuleC9300NM8X10G",
      "partNumber": "C9300-NM-8X",
      "isFieldReplaceable": "1"
    }
  ],
  "version": "1.0"
}
```

Teststatus

ERFOLGREICH

Tabelle 32: Ausführung T-11

T-12	Modul Abfrage mit inkorrekten Parametern (Parameter widersprechen sich)
TestszENARIO	Die Modul Abfrage wird mit Parametern ausgeführt, welche sich widersprechen.
Testvoraussetzung	Der Token ist korrekt und gültig. Der Parameter deviceId wurde mitgegeben und ist gültig.
Testschritte	1. Widersprechende Parameter eingeben. «?deviceId=075e39d1-976a-49e8-ab41-3ee908d504ef&partNumberList=C9300-NM-8X&vendorEquipmentTypeList=nichtexistent»
Erwartete Resultate	Die Abfrage liefert keine Module.
Output	
{ "response": [], "version": "1.0" }	
Teststatus	ERFOLGREICH

Tabelle 33: Ausführung T-12

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **97** von **129**

T-13	Events können gesendet werden (ohne Parameter)
TestszENARIO	Wenn Events ohne Parameter gesendet werden, wird ein zufälliger Event generiert und an die URL, welche im Konfigurationsfile hinterlegt ist, gesendet.
Testvoraussetzung	Beide Server Systeme sind in Betrieb.
Testschritte	1. Das File namens sendEvents.php mit «php sendEvents.php» im Terminal aufrufen.
Erwartete Resultate	Der Event wird auf der zweiten virtuellen Maschine, welcher die Seite des Users darstellt in die Events Tabelle geschrieben.
Output Einige Aufrufe wurden korrekt, Andere gar nicht in die Datenbank geschrieben.	
Teststatus	TEILWEISE FEHLGESCHLAGEN

Tabelle 34: Ausführung T-13

T-14	Events können gesendet werden (mit Parameter)
TestszENARIO	Events können selber erfasst werden und an die URL, welche im Konfigurationsfile hinterlegt ist, gesendet werden. Wenn der Nutzer beim File Aufruf nicht alle Parameter mitgibt, können diese nachträglich über das Terminal eingegeben werden.
Testvoraussetzung	Beide Server Systeme sind in Betrieb.
Testschritte	1. Das File namens sendEvents.php mit «php sendEvents.php -c» im Terminal aufrufen. 2. Die geforderten Parameter schrittweise eingeben.
Erwartete Resultate	Der Event wird mit den eingegebenen Informationen in die Datenbank geschrieben.
Output Einige Aufrufe wurden korrekt andere gar nicht in die Datenbank geschrieben.	
Teststatus	TEILWEISE FEHLGESCHLAGEN

Tabelle 35: Ausführung T-14

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **98** von **129**

Das Fazit nach dem erstmaligen Durchführen der Tests ist positiv. Alle Funktionen konnten korrekt umgesetzt werden. Allerdings ist bei Ausführung T-09 eine falsche Rückmeldung aufgefallen. Die Funktionalität der Abfrage wurde dabei nicht verfälscht.

Wie im Arbeitsjournal von Tag 7 erwähnt, funktioniert das Senden der Events nicht immer korrekt. Auch hier funktioniert das Geforderte teilweise. Dieser Fehler wird in der Überarbeitung noch einmal aufgegriffen.

19.3 Fehlerhafte Test Cases überarbeiten

Die fehlgeschlagenen Test Cases wurden überarbeitet und werden nun noch einmal getestet.

Der Test T-09 hat einen falschen Rückgabewert ergeben. Bei einer inkorrekten Eingabe der deviceId wurde lediglich gesagt, dass die Abfrage keinen Output liefert. Stattdessen sollte das Programm erkennen, dass die mitgegebene Id falsch ist und daher kein Modul gefunden wird. Nachdem der Output geändert wurde, war der Fehler behoben.

Der Test T-09 wurde mit denselben Schritten erneut getestet.

Der Test liefert den korrekten Output. Somit gilt der Test als **ERFOLGREICH**.

```
{ "response": { "errorCode": "Bad request", "message": "Invalid input request", "detail": "12345nichtexistent12345" is not a valid UUID of device" }, "version": "1.0" }
```

Code 44: Result Retest T-09

Da die fehlschlagende Ausführung von T-13 und T-14 auf denselben Fehler zurückzuführen ist, wird dieser als ein Fehler behandelt.

Es ist auffällig, dass nur einige Eventtypen nicht funktionieren. Die Eventtypen 7, 5, 4 und 1 funktionieren wie auf Abbildung 44 ersichtlich, einwandfrei. Weil allerdings Eventtypen alle mit der gleichen Schleife ausgelesen werden, ist dieser Fehler sehr merkwürdig.

dnaName	dnaDescription	dnaType	dnaCategory	dnaDomain	dnaSubDomain	dnaTimestamp	dnaDetails
Router unreachable	Device is unreachable	NETWORK	ALERT	domain	subDomain	2020-04-20 14:56:13	7
AP Memory High ...	AP is experiencing high ...	NETWORK	WARN	domain	subDomain	2020-04-20 14:56:06	5
Switch Fan Failure	Fan failure on switch	NETWORK	ERROR	domain	subDomain	2020-04-20 14:56:02	4
Switch Power Fail...	Power supply failure on ...	NETWORK	ERROR	domain	subDomain	2020-04-20 14:55:50	1

Abbildung 44: Funktionierende Events

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **99** von **129**

Als Erstes habe ich in der Datenbanktabelle, in der die Eventtypen festgehalten sind, die Eventtypen auf die korrekte Schreibweise kontrolliert. Es waren keine Schreibfehler oder Leerzeichen davor oder danach vorhanden. Somit liegt der Fehler sicherlich nicht beim Herauslesen der Dateien.

Um den Fehler im Code zu finden, wird eine Debugging Funktion gebaut, welche über den Parameter «-x» aufgerufen werden kann. Der logischere Parameter «-d» wurde bereits für die Details verwendet und kann daher nicht noch einmal verwendet werden.

```
mifehr@michi:/var/www/ipa_dna_simulator/events$ php sendEvents.php -x
generated EventId = NETWORK-DEVICES-2-201
generated Detail = detail 4
generated Severity = 4
generated Sql Tag = SELECT * from events where EventId='NETWORK-DEVICES-2-201'
Übergabe Array :
{"name":"Switch Power Failure","instanceId":"instanceId","eventId":"NETWORK-DEVICES-2-201","type":
"NETWORK","details":"detail 4","namespace":"ASSURANCE","domain":"domain","severity":4,"description
":"Power supply failure on switch","tags":"tags","category":"ERROR","subDomain":"subDomain"}
Answer from API :
{"status":1,"status_message":"Event registered Successfully."}
```

Abbildung 45: Debugging Events

Mithilfe der Debugging Funktion, werden die zufällig ausgewählten Werte ausgegeben. Zudem wird das Übergabe Array wie auch die Antwort des Empfänger Scripts ersichtlich.

Weil bei den funktionierenden und den nicht funktionierenden Einträgen die Ausgabe jeweils dieselbe ist, wird der Fehler nicht auf der Seite des sendEvents.php Scripts vermutet.

Um die korrekte Sendung der Events zu veranschaulichen, wird dem Empfängerfile ein Log hinzugefügt. Bei jedem Aufruf des sendEvents.php Files, wird der Sql Tag in ein Logfile geschrieben. (Group, 2020)

```
1. $fx = fopen('logs/events.log', 'a');
2. fwrite($fx, $sql)."\n";
3. fclose($fx);
```

Abbildung 46: events.log

Der Event Typ mit der Id 7 funktioniert jedes Mal. Der Event Typ mit der Id 0 dagegen nicht. Beide Eventtypen werden nun manuell abgesendet. Danach können die übermittelten Daten im events.log File eingesehen werden. Wenn diese Daten gleich sind, zeigt dies, dass die Funktion, welche die Events sendet, korrekt funktioniert.

```
mifehr@michi:/var/www/ipa_dna_simulator/events$ php sendEvents.php -c
Dear user,
You have to choose which Eventtype you want to send.
Please choose one of the following Eventtypes by number:
0 | NETWORK-DEVICES-2-152 | WLC Reboot Crash |
1 | NETWORK-DEVICES-2-201 | Switch Power Failure |
2 | NETWORK-DEVICES-2-202 | Device Reboot Crash |
3 | NETWORK-DEVICES-2-204 | Stack Member Running Incompatible Image |
4 | NETWORK-DEVICES-2-205 | Switch Fan Failure |
5 | NETWORK-DEVICES-3-104 | AP Memory High Utilization |
6 | NETWORK-FABRIC_WIRED-1-310 | Fabric Devices Connectivity Control Underlay |
7 | NETWORK-NON-FABRIC_WIRED-1-250 | Router unreachable |
7
Please write some details about the choosen event:
7
Please provide the severity
1 | Info |
2 | Warning |
3 | Alert |
4 | Critical |
1
The event will be sent to http://inventarmichi/receiveEvents
```

```
mifehr@michi:/var/www/ipa_dna_simulator/events$ php sendEvents.php -c
Dear user,
You have to choose which Eventtype you want to send.
Please choose one of the following Eventtypes by number:
0 | NETWORK-DEVICES-2-152 | WLC Reboot Crash |
1 | NETWORK-DEVICES-2-201 | Switch Power Failure |
2 | NETWORK-DEVICES-2-202 | Device Reboot Crash |
3 | NETWORK-DEVICES-2-204 | Stack Member Running Incompatible Image |
4 | NETWORK-DEVICES-2-205 | Switch Fan Failure |
5 | NETWORK-DEVICES-3-104 | AP Memory High Utilization |
6 | NETWORK-FABRIC_WIRED-1-310 | Fabric Devices Connectivity Control Underlay |
7 | NETWORK-NON-FABRIC_WIRED-1-250 | Router unreachable |
0
Please write some details about the choosen event:
0
Please provide the severity
1 | Info |
2 | Warning |
3 | Alert |
4 | Critical |
1
The event will be sent to http://inventarmichi/receiveEvents
```

Abbildung 47: Ausführung Event Typen 0 und 7

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile
E-Mail
Security

079 355 16 94
Michael.Fehr@Swisscom.com
C2

Datum: 22.04.2020
Seiten: 100 von 129

1. INSERT INTO events (dnaId,dnaEventId,dnaNamespace,dnaName,dnaDescription,dnaType,dnaCategory,dnaDomain,dnaSubDomain,dnaSeverity,dnaTimestamp,dnaDetails,dnaTags,ipaddress,customerId) VALUES ('instanceId','NETWORK-NON-FABRIC_WIRED-1-250','ASSURANCE','Router unreachable','Device is unreachable','NETWORK','ALERT','domain','subDomain','1','2020-04-20 17:07:58','7','tags','192.168.56.102','0')

Code 45: Output Event Typ 7

2. INSERT INTO events (dnaId,dnaEventId,dnaNamespace,dnaName,dnaDescription,dnaType,dnaCategory,dnaDomain,dnaSubDomain,dnaSeverity,dnaTimestamp,dnaDetails,dnaTags,ipaddress,customerId) VALUES ('instanceId','NETWORK-DEVICES-2-152','ASSURANCE','WLC Reboot Crash','WLC has rebooted due to a hardware or software crash','NETWORK','ERROR','domain','subDomain','1','2020-04-20 17:09:57','0','tags','192.168.56.102','0')

Code 46: Output Event Typ 0

Auf dem Code 45 und dem Code 46 sind die Outputs der jeweiligen Event Typen ersichtlich. Beide sind bis auf die eingetragenen Informationen (Event Typ 7 oder 0 und Detail 7 oder 0) identisch. Das zeigt, dass mein Code, der während der IPA erstellt wurde, korrekt und funktionsfähig ist.

Swisscom (Schweiz) AG

Michael Fehr, GHR-OSA-NEX-33
Wassergasse 42
9000 St. Gallen

Mobile 079 355 16 94
E-Mail Michael.Fehr@Swisscom.com
Security C2

Datum: 22.04.2020
Seiten: **101** von **129**